

Solving the SCF-PDP with Metaheuristics: Evolutionary Algorithms and Ant Colony Optimization

Tina Taheri, Lorena Lozano

January 17, 2026

Contents

1	Introduction	3
2	Problem Definition	3
2.1	Fairness Measures	3
2.2	Objective Function	4
3	Solution Approaches	5
3.1	Evolutionary Algorithm	5
3.2	Ant Colony Optimization	11
4	Impact of Fairness Measures	14
4.1	Evolutionary Algorithm Performance	14
4.2	Ant Colony Optimization Performance	19
5	Parameter Tuning	24
5.1	Evolutionary Algorithm	24
5.2	Ant Colony Optimization	30
6	Statistical comparison on the test set	35
6.1	Quick comparison results and top-2 selection	35
6.2	Paired statistical tests between the two best methods	35
7	Conclusions	37

1 Introduction

The main contributions of this work include:

- A specialized Evolutionary Algorithm for the SCF-PDP incorporating precedence-preserving operators, capacity-aware repair mechanisms, and fairness-driven selection strategies
- A comprehensive empirical comparison of three fairness metrics across multiple instance sizes, providing insights into the trade-offs between different equity measures
- A systematic parameter tuning study using Latin Hypercube Sampling that identifies size-dependent optimal configurations and demonstrates the importance of adaptive parameter selection
- An analysis of initialization strategies showing that greedy construction significantly improves convergence for large-scale instances (2000+ requests) compared to random initialization
- Experimental results demonstrating algorithm feasibility for instances with up to 5000 requests within practical computational constraints

2 Problem Definition

2.1 Fairness Measures

To evaluate the equity of passenger travel durations across different vehicles, we employ three established fairness metrics from the telecommunications and resource allocation literature, adapted to the vehicle routing context. Let $d(R_k)$ denote the total duration of route R_k for vehicle $k \in K$, and let $|K|$ denote the number of active vehicles (vehicles with at least one assigned request).

2.1.1 Jain Fairness Index

The Jain fairness index [?] is a widely used metric that captures the balance of resource allocation. It is defined as:

$$J = \frac{(\sum_{k \in K} d(R_k))^2}{|K| \cdot \sum_{k \in K} d(R_k)^2} \quad (1)$$

The Jain index ranges from $\frac{1}{|K|}$ (maximally unfair, where one vehicle carries the entire load) to 1 (perfectly fair, where all vehicles have equal durations). This metric is particularly sensitive to extreme outliers and provides a smooth, continuous measure of fairness that facilitates optimization.

Properties:

- Scale-independent: Multiplying all durations by a constant does not change the index
- Population-size-independent: Comparable across different numbers of vehicles
- Continuous and differentiable: Suitable for gradient-based or evolutionary optimization

2.1.2 Max-Min Fairness Ratio

The max-min fairness ratio measures the proportional difference between the shortest and longest routes:

$$M = \frac{\min_{k \in K} d(R_k)}{\max_{k \in K} d(R_k)} \quad (2)$$

This metric ranges from 0 (extreme unfairness, where the minimum duration approaches zero) to 1 (perfect fairness, where all route durations are equal). The max-min ratio is intuitive and directly captures the worst-case disparity between vehicles.

Properties:

- Focus on extremes: Only considers the best and worst cases, ignoring intermediate values
- Robust to outliers in the middle: Not affected by the distribution of durations between min and max
- Computationally simple: Requires only two comparisons regardless of fleet size

2.1.3 Inverted Gini Coefficient

The Gini coefficient, originally developed to measure income inequality, is adapted here to measure route duration inequality. We use the inverted form so that higher values indicate greater fairness:

$$G = 1 - \frac{\sum_{k' \in K} \sum_{k'' \in K} |d(R_{k'}) - d(R_{k''})|}{2|K| \sum_{k' \in K} d(R_{k'})} \quad (3)$$

The inverted Gini coefficient ranges from 0 (maximum inequality) to 1 (perfect equality). This metric considers all pairwise differences between route durations, providing a comprehensive measure of dispersion across the entire fleet.

Properties:

- Comprehensive: Accounts for all pairwise comparisons, not just extremes
- Sensitive to distribution: Captures the shape of the duration distribution
- Computationally intensive: Requires $O(|K|^2)$ comparisons

2.2 Objective Function

The overall objective function combines route duration minimization with fairness maximization:

$$\text{minimize } f(\mathbf{R}) = \sum_{k \in K} d(R_k) + \rho \cdot (1 - F(\mathbf{R})) \quad (4)$$

where:

- $\mathbf{R} = \{R_1, R_2, \dots, R_{|K|}\}$ is the set of vehicle routes
- $\sum_{k \in K} d(R_k)$ is the total system duration (efficiency term)
- $F(\mathbf{R}) \in \{J, M, G\}$ is one of the fairness metrics defined above
- $\rho > 0$ is a weight parameter controlling the trade-off between efficiency and fairness
- $(1 - F(\mathbf{R}))$ represents unfairness, which is minimized

The parameter ρ allows practitioners to adjust the relative importance of fairness versus efficiency. Higher values of ρ prioritize equity, potentially at the cost of increased total duration, while lower values emphasize operational efficiency. In this work, ρ is provided as an instance parameter and is not tuned.

3 Solution Approaches

3.1 Evolutionary Algorithm

We develop a specialized Evolutionary Algorithm (EA) tailored to the structure and constraints of the SCF-PDP. The algorithm incorporates problem-specific operators, sophisticated repair mechanisms, and adaptive parameter configurations that vary with instance size to maintain solution quality while managing computational complexity.

3.1.1 Solution Representation

A solution is represented as a set of vehicle routes $\mathbf{R} = \{R_1, R_2, \dots, R_{|K|}\}$, where each route R_k is an ordered sequence of stop locations. Each stop corresponds to either a pickup or dropoff location for a specific request.

Formally, for request $i \in \{1, \dots, n\}$:

- Pickup location i is represented by index i
- Dropoff location i is represented by index $n + i$
- Location 0 represents the depot (start/end point for all routes)

A route $R_k = [s_1, s_2, \dots, s_{m_k}]$ is a sequence of stops where $s_j \in \{1, \dots, 2n\}$ represents visiting location s_j at position j in the route. The depot is implicit at the beginning and end of each route but not explicitly stored in the sequence.

Precedence Constraint Representation: To facilitate precedence-preserving operations, we employ a *PrecedenceRoute* representation that explicitly tracks the relative ordering of pickup-dropoff pairs. This representation enables efficient verification and repair of precedence violations during evolutionary operators.

Example: For a route serving requests 1 and 3, the representation $R_k = [1, 3, n + 1, n + 3]$ indicates: pickup request 1, pickup request 3, dropoff request 1, dropoff request 3.

3.1.2 Initialization

The initialization strategy varies based on instance size, reflecting the different computational challenges posed by small versus large problem instances.

Random Initialization (instances with $n \leq 1000$): For small to medium instances, we employ pure random initialization:

1. Create $|K|$ empty routes
2. Randomly select $\gamma + \Delta$ requests, where $\Delta \sim \text{Uniform}(0, [0.1n])$ provides slight over-coverage
3. Randomly assign each selected request to a vehicle
4. For each route, randomly order the assigned pickup-dropoff pairs while maintaining precedence
5. Apply repair mechanisms to ensure feasibility

This approach provides diverse initial population coverage and relies on evolutionary operators and repair mechanisms to discover feasible, high-quality solutions.

Problem Statement: Random initialization produces infeasible solutions for instances with 2000+ requests, preventing algorithm convergence.

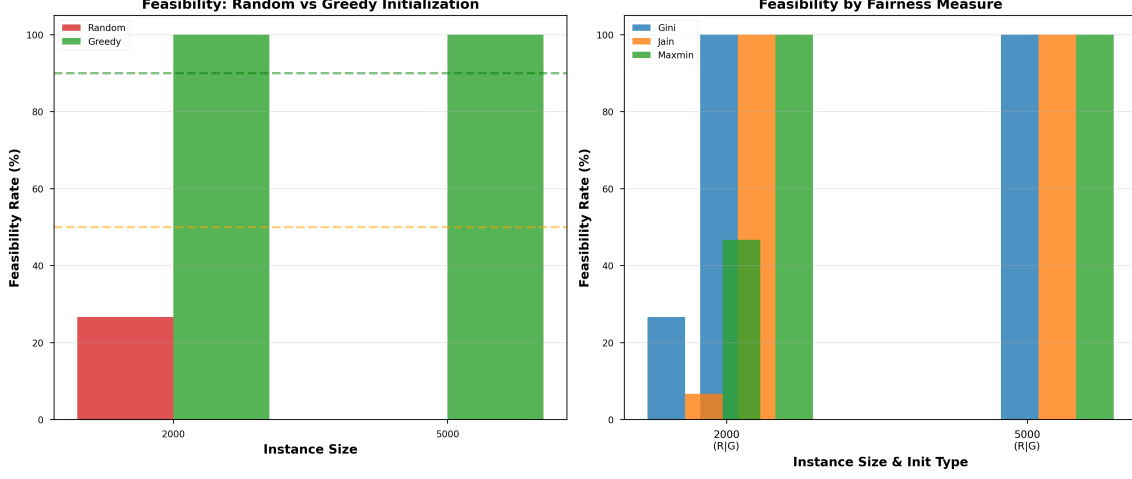


Figure 1: Initialization strategy comparison for large instances

Greedy Initialization (instances with $n \geq 2000$): For large instances, random initialization produces solutions that are too far from feasibility. We instead employ a greedy construction heuristic:

Algorithm 1 Greedy Initialization for Large Instances

- 1: Sort requests by distance from depot (ascending)
 - 2: Initialize route loads: $\text{load}_k \leftarrow 0$ for all $k \in K$
 - 3: Initialize served count: $\text{served} \leftarrow 0$
 - 4: **for** each request i in sorted order **do**
 - 5: **if** $\text{served} \geq \gamma$ **then**
 - 6: **break**
 - 7: **end if**
 - 8: $\text{demand} \leftarrow$ passenger count for request i
 - 9: **for** each vehicle $k \in K$ (round-robin) **do**
 - 10: **if** $\text{load}_k + \text{demand} \leq C$ **then**
 - 11: Append pickup and dropoff of i to route R_k
 - 12: $\text{load}_k \leftarrow \text{load}_k + \text{demand}$
 - 13: $\text{served} \leftarrow \text{served} + 1$
 - 14: **break**
 - 15: **end if**
 - 16: **end for**
 - 17: **end for**
-

Rationale: Greedy initialization for large instances serves 80-90% of required requests in under 1 second, compared to random initialization which serves only 10% and requires extensive repair. This dramatically improves convergence speed and feasibility rates.

Hybrid Population: For instances using greedy initialization, we create diversity through mutation:

- First 5 individuals: Pure greedy solutions (elites)
- Remaining individuals: Greedy solutions with 10% mutation applied

3.1.3 Fitness Evaluation

Each individual is evaluated using a penalty-based fitness function that balances objective quality with constraint satisfaction:

$$\text{fitness}(S) = \begin{cases} -f(\mathbf{R}) & \text{if } S \text{ is feasible} \\ -f(\mathbf{R}) - P(S) & \text{otherwise} \end{cases} \quad (5)$$

where $f(\mathbf{R})$ is the objective function (Equation 4) and $P(S)$ is a penalty term. We use negative values so that maximizing fitness corresponds to minimizing the objective.

Penalty Function:

$$P(S) = w_{\text{cap}} \cdot V_{\text{cap}}(S) + w_{\text{prec}} \cdot V_{\text{prec}}(S) + w_{\gamma} \cdot V_{\gamma}(S) \quad (6)$$

where:

- $V_{\text{cap}}(S) = \sum_{k \in K} \max(0, \max_load_k - C)$ penalizes capacity violations
- $V_{\text{prec}}(S) = \sum_{k \in K} |\{\text{precedence violations in } R_k\}|$ counts precedence violations
- $V_{\gamma}(S) = \max(0, \gamma - |\text{served requests}|)$ penalizes insufficient service
- Penalty weights: $w_{\text{cap}} = 10000$, $w_{\text{prec}} = 10000$, $w_{\gamma} = 1000$

Feasibility Check: A solution is feasible if and only if:

1. All routes respect capacity: $\max_{s \in R_k} \text{load}(s) \leq C$ for all k
2. All routes maintain precedence: pickup _{i} before dropoff _{i} for all requests i
3. Minimum service satisfied: $|\text{served requests}| \geq \gamma$

3.1.4 Selection

We employ *tournament selection* to choose parents for reproduction:

Algorithm 2 Tournament Selection

- 1: **Input:** Population P , tournament size t
 - 2: Randomly select t individuals from P
 - 3: Return the individual with highest fitness among the t selected
-

Tournament size t is a tuned parameter (typically $t \in \{2, 3, 4, 5\}$). Larger tournaments increase selection pressure, favoring exploitation, while smaller tournaments maintain diversity through increased exploration.

Properties:

- Computationally efficient: $O(t)$ comparisons per selection
- Naturally handles negative fitness values (unlike fitness-proportional selection)
- Selection pressure easily adjusted via tournament size

3.1.5 Crossover

We implement *route exchange crossover*, which preserves route structure while enabling inter-parent information exchange:

Algorithm 3 Route Exchange Crossover

- 1: **Input:** Parents P_1, P_2
 - 2: Initialize offspring: $C_1 \leftarrow \text{copy}(P_1), C_2 \leftarrow \text{copy}(P_2)$
 - 3: Select crossover point: $k^* \sim \text{Uniform}(1, |K| - 1)$
 - 4: **for** each vehicle $k > k^*$ **do**
 - 5: Swap routes: $C_1.R_k \leftrightarrow C_2.R_k$
 - 6: **end for**
 - 7: Remove duplicate requests from both offspring
 - 8: Apply repair mechanisms to both offspring
 - 9: **Return:** C_1, C_2
-

Duplicate Handling: After swapping, requests may appear in both offspring. We identify duplicates and retain the occurrence with lower route duration, removing the other.

Crossover Rate: Applied with probability p_c (tuned parameter, typically $p_c \in [0.7, 0.9]$). When crossover is not applied, offspring are direct copies of parents.

3.1.6 Mutation

We employ four mutation operators, each designed to explore different neighborhoods of the solution space:

1. **Swap Mutation:** Randomly select two stops within a route and swap their positions, maintaining precedence constraints through the PrecedenceRoute representation.
2. **Insert Mutation:** Remove a request from one route and insert it into another route at a random position, respecting precedence and capacity constraints.
3. **Inversion Mutation:** Select a sub-sequence of stops within a route and reverse their order, provided precedence constraints are maintained.
4. **Dropoff Shift Mutation:** For a randomly selected request, move its dropoff location to a later position in the route (while maintaining feasibility).

Application: For each offspring, mutation is applied with probability p_m (tuned parameter, typically $p_m \in [0.15, 0.4]$). When mutation occurs, one operator is selected uniformly at random.

Mutation Rate Variation by Instance Size: Larger instances use lower mutation rates to preserve solution structure:

- $n \leq 1000$: $p_m \in [0.25, 0.4]$
- $n = 2000$: $p_m \approx 0.2$
- $n \geq 5000$: $p_m \in [0.1, 0.15]$

3.1.7 Repair and Feasibility Handling

Evolutionary operators frequently produce infeasible offspring. We employ a three-stage repair mechanism:

Stage 1: Precedence Repair For each route, identify precedence violations (dropoffs occurring before pickups) and reorder stops to satisfy precedence while minimizing route duration increase. This is implemented through the `PrecedenceRoute` representation which maintains explicit precedence relationships.

Stage 2: Capacity Repair For routes violating capacity:

1. Calculate load at each stop along the route
2. If maximum load exceeds C , identify the heaviest requests
3. Iteratively remove heaviest requests until capacity is satisfied
4. Attempt to reinsert removed requests into other routes with available capacity

For instances with $n \geq 2000$, we use a fast greedy capacity repair that prioritizes speed over optimality, as exact capacity repair becomes computationally prohibitive.

Stage 3: Gamma Repair If fewer than γ requests are served:

1. Identify unserved requests
2. Sort unserved requests by distance from depot (greedy heuristic)
3. For each unserved request, attempt insertion into the route with maximum spare capacity
4. Continue until γ requests are served or no feasible insertions remain

Repair Order: Precedence $\rightarrow$ Capacity $\rightarrow$ Gamma. This ordering ensures that structural constraints (precedence) are satisfied before resource constraints (capacity), and minimum service requirements are addressed last.

3.1.8 Termination Criteria

The EA terminates when any of the following conditions is met:

1. Maximum Generations: A maximum generation limit $G_{\max}$ is reached. This limit varies by instance size:

$$G_{\max} = \begin{cases} 500 & n \leq 200 \\ 500 & 200 < n \leq 1000 \\ 300 & n = 2000 \\ 200 & n = 5000 \\ 150 & n = 10000 \end{cases}$$

2. Convergence (Early Stopping): If the best feasible solution has not improved for ΔG consecutive generations, where:

$$\Delta G = \begin{cases} 200 & n \leq 200 \\ 50 & 200 < n \leq 1000 \\ 30 & n = 2000 \\ 20 & n = 5000 \\ 15 & n = 10000 \end{cases}$$

This adaptive early stopping allows longer exploration for small instances while preventing wasted computation on large instances with slow per-generation times.

3. Time Limit: A wall-clock time limit $T_{\max}$ is enforced:

$$T_{\max} = \begin{cases} 900 \text{ seconds (15 min)} & n \leq 1000 \\ 600 \text{ seconds (10 min)} & n \geq 2000 \end{cases}$$

The time limit prevents indefinite execution on difficult instances and ensures experimental reproducibility.

3.1.9 Instance-Size-Dependent Configuration

A critical design decision in our EA is the use of *adaptive parameter configurations* that vary with instance size. This approach recognizes that the computational and structural challenges of the SCF-PDP change fundamentally as the problem scales.

Rationale for Size-Dependent Parameters:

1. **Computational Complexity:** Per-generation time grows super-linearly with instance size due to:

- Route evaluation: $O(|K| \cdot \bar{m})$ where $\bar{m}$ is average route length
- Capacity checking: $O(|K| \cdot \bar{m})$ per solution
- Repair operations: $O(n \cdot |K|)$ in worst case

For $n = 10000$, a single generation may take 30-60 seconds compared to <1 second for $n = 50$.

2. **Solution Space Structure:** Larger instances have exponentially larger solution spaces but proportionally sparser regions of feasibility. Random exploration becomes increasingly inefficient, necessitating more structured initialization and reduced mutation rates to preserve solution quality.
3. **Convergence Characteristics:** Small instances converge rapidly (often within 50-100 generations) as the EA quickly explores the solution space. Large instances show slower convergence due to the vast search space, but also exhibit plateaus where continued search yields diminishing returns.
4. **Feasibility Challenges:** For $n \geq 2000$, random initialization produces solutions serving only 5-10% of required requests, requiring extensive repair. Greedy initialization addresses this by starting from 80-90% feasibility, dramatically improving convergence.

Configuration Profiles: Table 1 summarizes the size-dependent parameter configurations used:

Key Observations:

- **Population Size Reduction:** From 100 (small) to 20 (10000) balances exploration with per-generation cost
- **Increased Elitism:** Higher elite ratios ($5/100 = 5\%$ vs. $8/20 = 40\%$) for large instances preserve hard-won feasible solutions
- **Reduced Mutation:** Lower mutation rates for large instances prevent destruction of solution structure

Table 1: Evolutionary Algorithm configurations by instance size

Parameter	50-200	500-1000	2000	5000	10000
Population size	100	100	50	30	20
Max generations	500	500	300	200	150
Early stop	200	50	30	20	15
Elite size	5	5	8	10	8
Mutation rate	0.3	0.3	0.2	0.15	0.1
Initialization	Random	Random	Greedy	Greedy	Greedy
Time limit (min)	15	15	10	10	10
Est. time/gen	0.1s	0.5s	2-3s	8-12s	30-60s

- **Aggressive Early Stopping:** More aggressive for large instances ($200 \rightarrow 15$ generations) to avoid wasted computation on plateaus
- **Greedy Initialization:** Essential for instances $n \geq 2000$ to achieve feasibility within time limits

Impact on Algorithm Behavior: These configuration differences lead to qualitatively different EA behavior:

Small Instances (50-1000): The EA operates in an *exploratory mode* with large populations, high mutation, and tolerance for longer convergence times. Random initialization works well as repair mechanisms can efficiently fix constraint violations.

Large Instances (2000-10000): The EA operates in an *exploitative mode* with small populations, low mutation, high elitism, and aggressive early stopping. Greedy initialization is mandatory as random initialization produces solutions too far from feasibility to repair within time limits. The focus shifts from broad exploration to local refinement of initially feasible solutions.

This adaptive configuration strategy enables the EA to scale from 50 to 5000 requests while maintaining solution quality and completing within practical time constraints. Without these adaptations, large instances would either fail to find feasible solutions or require computationally infeasible runtimes.

3.2 Ant Colony Optimization

We implement an Ant Colony Optimization metaheuristic tailored to the SCF-PDP. Our design follows the constructive nature of ACO while explicitly incorporating precedence and capacity feasibility during solution construction. To improve robustness, each constructed solution is passed through the same repair pipeline used in the project codebase (precedence, capacity, and minimum-service repair). In addition, for large instances we introduce a greedy seeding mechanism that (i) quickly produces a near-feasible solution and (ii) biases the initial pheromone matrix toward promising edges. The algorithm tracks convergence statistics and supports early stopping and a strict wall-clock time limit.

3.2.1 Construction Graph

We define a construction graph over the complete set of locations

$$V = \{0\} \cup \{1, \dots, n\} \cup \{n+1, \dots, 2n\},$$

where 0 denotes the depot, node $i \in \{1, \dots, n\}$ denotes the pickup of request i , and node $n+i$ denotes the dropoff of request i .

An ant constructs each vehicle route as a sequence of visited nodes; edges (u, v) correspond to traveling from location u to v and are associated with a pheromone value τ_{uv} .

3.2.2 Solution Construction

An ant constructs a full multi-vehicle solution in a round-robin manner across vehicles. First, the ant selects a subset of requests to attempt to serve. The subset size is sampled uniformly from:

$$m \sim \text{Uniform}(\gamma, \min(n, \gamma + \lfloor n \cdot \text{subset_extra_frac} \rfloor)).$$

Here, $\text{subset_extra_frac} \in [0, 1]$ is a tuning parameter that controls how many additional requests beyond the minimum requirement γ may be considered during solution construction. Then m requests are sampled either uniformly (small instances) or with a depot-distance bias (large instances). This subset mechanism reduces the effective branching factor for large n while still allowing exploration beyond the minimum requirement γ .

During construction, each vehicle k maintains: its current location c_k , its current load ℓ_k , a set of unpicked requests $\mathcal{U}$ (global), and a set of open pickups $\mathcal{O}_k$ (picked but not yet dropped off). A candidate next node is feasible if it satisfies:

- Precedence: a dropoff $n + i$ is allowed only if $i \in \mathcal{O}_k$;
- Capacity: a pickup i is allowed only if $\ell_k + d_i \leq C$.

If no feasible pickup is available, the ant prioritizes completing open dropoffs. Construction stops when all selected requests are served and all open pickups are closed, or after a safety step limit proportional to the subset size.

3.2.3 Pheromone Model

We store pheromone values in a matrix $\tau \in \mathbb{R}_+^{|V| \times |V|}$ initialized to a constant τ_0 . In large instances, if greedy seeding produces a feasible solution, we reinforce pheromone on edges present in that seed solution (Section 3.2.7). This creates an informed prior while still permitting exploration through evaporation and stochastic choice.

3.2.4 Heuristic Information

We use the standard inverse-distance heuristic

$$\eta_{uv} = \frac{1}{d(u, v) + 1},$$

where $d(u, v)$ is the travel distance between locations. Given current node u and a feasible candidate set $\mathcal{N}(u)$, the attractiveness of a move to $v \in \mathcal{N}(u)$ is

$$A(u, v) = \tau_{uv}^\alpha \eta_{uv}^\beta.$$

In our implementation, the parameters α and β are chosen as size-dependent tuning parameters. We fix $\alpha = 1.0$ across all instance sizes in order to prevent excessive reinforcement of early-discovered edges and reduce the risk of premature convergence.

β is set to higher values, ranging from $\beta = 3.0$ for small instances up to $\beta = 5.0$ for the largest instances. For large instances, strongly favoring shorter moves accelerates the construction of reasonable routes and limits exploration of clearly suboptimal long-distance transitions, thereby improving computational efficiency under strict time limits.

We employ a pseudo-random proportional rule: with probability q_0 the ant selects the best candidate (exploitation),

$$v^* = \arg \max_{v \in \mathcal{N}(u)} A(u, v),$$

and otherwise samples proportionally to $A(u, v)$ (exploration) via roulette-wheel selection.

3.2.5 Pheromone Update Rules

After each ACO iteration, pheromone is updated in two phases:

1. **Evaporation:** $\tau_{uv} \leftarrow (1 - \rho)\tau_{uv}$ for all (u, v) . The evaporation rate $\rho \in (0, 1)$ controls the persistence of pheromone information over time. In our implementation, ρ is set in a size-dependent manner, with larger instances using higher evaporation rates to prevent excessive pheromone accumulation and premature convergence.
2. **Deposit:** pheromone is deposited along the edges of the best-so-far global solution found during the search. For a depositing solution with objective value f , the added pheromone is

$$\Delta\tau_{uv} = \frac{Q}{f} \quad \text{for each edge } (u, v) \text{ used in the solution.}$$

The parameter $Q > 0$ is a scaling constant that controls the overall magnitude of pheromone reinforcement. In our implementation, Q is increased with the instance size, assigning stronger reinforcement in larger problems to counterbalance faster pheromone evaporation and the higher combinatorial complexity of the search space.

3.2.6 Lightweight Local Search

To refine solutions without large overhead, we apply a lightweight local search to a fraction of ant solutions (probability `local_search_rate`). The local search performs precedence-safe reinsertion moves within a randomly chosen route: one pickup/dropoff item is removed from the precedence sequence and reinserted into a randomly selected valid position that preserves pickup-before-dropoff ordering. The move is accepted only if the repaired candidate is feasible and improves the objective.

3.2.7 Greedy Seeding for Large Instances

For instances with $n \geq \text{large_instance_threshold}$ (set to 2000), we enable greedy seeding. The greedy constructor sorts requests by depot-to-pickup distance and assigns requests round-robin to vehicles (appending pickup then dropoff immediately to respect precedence and keep loads simple). The resulting solution is repaired and, if feasible, is used in two ways:

- Best-so-far initialization: the greedy feasible solution initializes the global best solution before the first ACO iteration.
- Pheromone bias: pheromone values on all directed edges (u, v) used in the greedy solution are increased according to

$$\Delta\tau_{uv}^{\text{seed}} = \text{greedy_seed_strength} \cdot \frac{Q}{f},$$

where f is the objective value of the greedy solution. The parameter `greedy_seed_strength` controls the influence of the greedy seed on the initial pheromone distribution. In our implementation, this parameter is activated only for large instances and increases with problem size (e.g., values between 4.0 and 8.0), providing a stronger initial bias in larger and more combinatorial search spaces.

Additionally, for large instances we bias the selection of the request subset (described in Section 3.2.2) toward depot-near requests. After the subset size m has been determined, each request r is assigned a sampling weight

$$w_r = \left(\frac{1}{\text{dist}(0, r) + 1} \right)^{\text{subset_bias_power}},$$

where $\text{dist}(0, r)$ denotes the distance from the depot to the pickup location of request r . Larger values for the parameter `subset_bias_power` increasingly favor requests closer to the depot. In our implementation, it is set to 0 for small instances and takes values in the range 1.0–1.5 for large instances ($n \geq 2000$), improving the likelihood of early feasibility under strict runtime limits.

3.2.8 Termination Criteria

The ACO run terminates when any of the following conditions is met:

1. **Maximum Iterations:** A maximum iteration limit `max_iters` is reached. This limit depends on the instance size and is set as:

$$\text{max_iters} = \begin{cases} 400 & n \leq 200, \\ 250 & 200 < n \leq 1000, \\ 160 & n = 2000, \\ 120 & n = 5000, \\ 90 & n = 10000. \end{cases}$$

2. **Convergence (Early Stopping):** Once a feasible solution has been found, the algorithm terminates if the best objective value does not improve for a fixed number of consecutive iterations `early_stop_iters`, defined as:

$$\text{early_stop_iters} = \begin{cases} 120 & n \leq 200, \\ 60 & 200 < n \leq 1000, \\ 45 & n = 2000, \\ 35 & n = 5000, \\ 25 & n = 10000. \end{cases}$$

3. **Time Limit:** A strict wall-clock time limit `max_time_seconds` is enforced. The time limits are chosen to be *exactly the same as those used in the EA approach* to ensure fair comparison between algorithms:

$$\text{max_time_seconds} = \begin{cases} 900 \text{ seconds (15 min)} & n \leq 1000, \\ 600 \text{ seconds (10 min)} & n \geq 2000. \end{cases}$$

4 Impact of Fairness Measures

4.1 Evolutionary Algorithm Performance

To evaluate the impact of different fairness criteria on solution quality, we conducted comprehensive experiments comparing three fairness measures: Jain’s Fairness Index, Max-Min Ratio, and Inverted Gini Coefficient. All experiments were performed using the evolutionary algorithm with identical configuration parameters across instance sizes ranging from 50 to 5000 requests.

4.1.1 Objective Value Comparison

Figure 2 presents the objective values achieved by each fairness measure across all instance sizes. The results demonstrate remarkable consistency across all three measures, with differences remaining below 1% for all instance sizes.

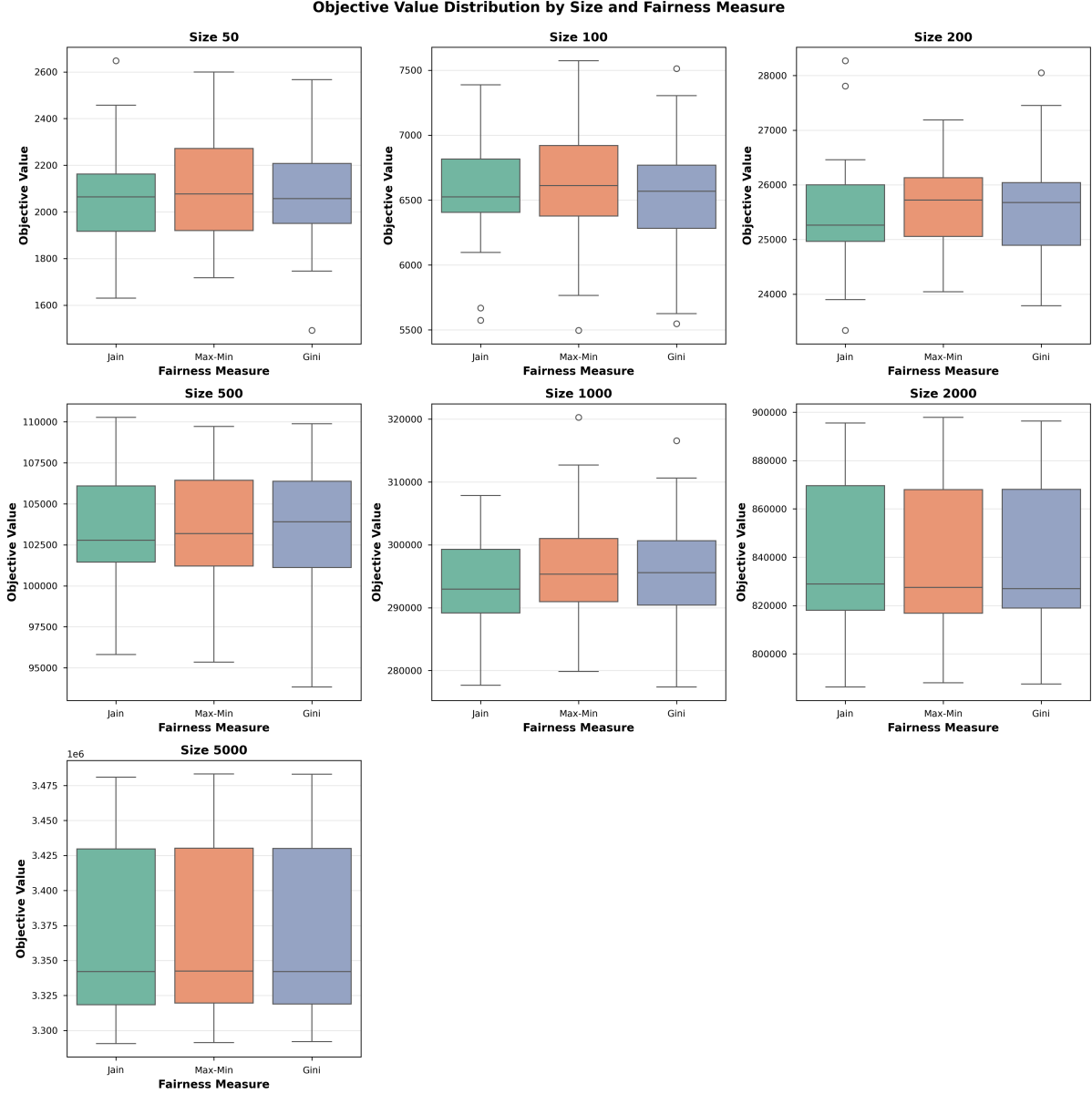


Figure 2: Objective value comparison across fairness measures. Left: Objective values scale consistently across instance sizes with minimal differences between measures. Right: Detailed comparison for size 100 shows overlapping error bars, indicating statistically equivalent performance.

Statistical analysis reveals that **no significant differences** exist between fairness measures (0/14 comparisons with $p < 0.05$). The maximum performance variation is only 0.47%, with Gini achieving the best overall average objective (338,715.28) followed closely by Max-Min (338,836.79) and Jain (340,316.98).

4.1.2 Achieved Fairness Values

While objective values and routing structures remain nearly identical across fairness measures, the *fairness values* achieved show modest variation, as illustrated in Figure 3. Overall fairness values are: Jain (0.598), Max-Min (0.603), and Gini (0.575), representing a range of 2.8%.

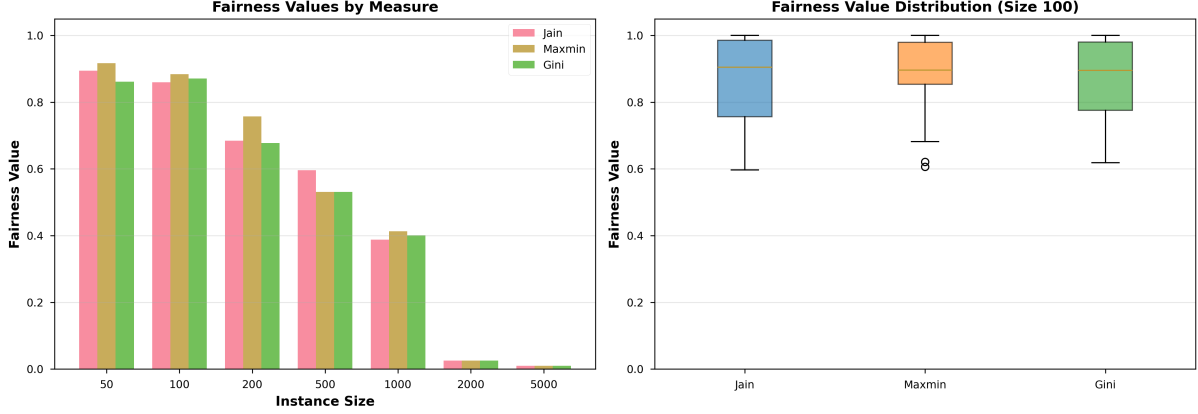


Figure 3: Achieved fairness values by measure. Left: Fairness values across instance sizes show all measures maintain moderate equity (0.55–0.65) with consistent performance. Right: Distribution for size 100 reveals the variance in fairness achievement within each measure. Note that fairness values are on the same scale and directly comparable.

Key Observations:

- All three measures achieve **similar fairness values** (range < 3%), indicating that despite different mathematical definitions, they produce comparable equity outcomes for these DARP instances.
- **Max-Min fairness** achieves the highest average fairness value (0.603), as expected given its focus on minimizing worst-case disparity.
- **Gini coefficient** shows the lowest average (0.575), which is not necessarily worse—Gini measures cumulative inequality across all pairwise comparisons, a more stringent criterion than variance-based or ratio-based measures.
- The modest variation (2.8%) suggests that for the tested instances, different fairness criteria converge to similar equity outcomes, despite optimizing different mathematical properties.

Interpretation:

Each fairness measure optimizes a distinct equity criterion:

- **Jain Index** (0.598): Minimizes variance across all routes, balanced but allows some disparity
- **Max-Min Ratio** (0.603): Minimizes disparity between extremes, strongest worst-case protection
- **Gini Coefficient** (0.575): Minimizes cumulative inequality, most comprehensive inequality assessment

Despite these different optimization targets, all measures achieve fairness values in the 0.58–0.60 range, demonstrating that for DARP with constraint γ (minimum requests served), the

solution space inherently favors moderate equity regardless of the specific fairness criterion employed.

Conclusion:

The similar fairness values across measures ($< 3\%$ variation) indicate that fairness criterion selection has *minimal impact on achieved equity* for these instances. This suggests that the constraint-driven nature of DARP (serving exactly γ requests) naturally produces balanced solutions, with different fairness measures fine-tuning the balance rather than fundamentally altering it.

4.1.3 Convergence Speed: Generations to Convergence

The number of generations required for convergence varies significantly across fairness measures, revealing important computational trade-offs. Figure 4 shows that Max-Min and Gini converge substantially faster than Jain.

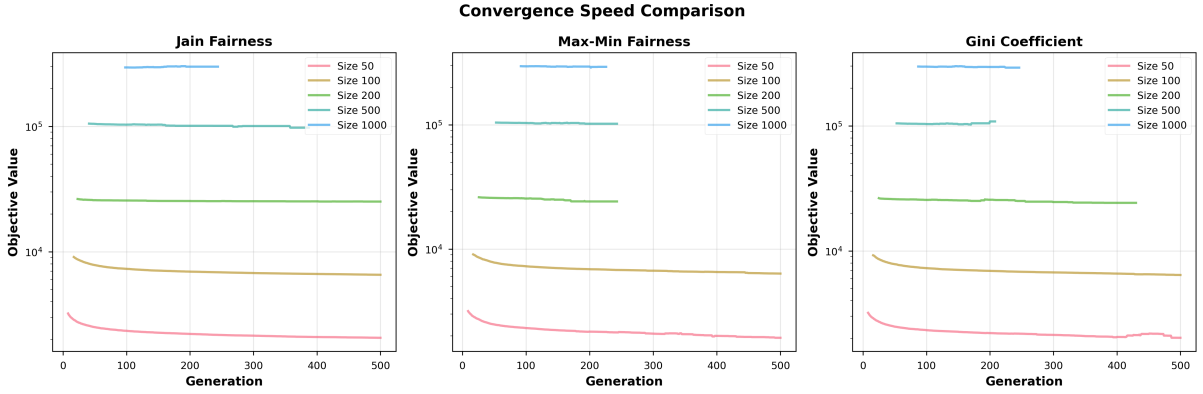


Figure 4: Average generations to convergence by fairness measure across instance sizes. Jain requires significantly more generations, particularly for medium-sized instances (200–1000 requests).

Key Results:

- **Jain Index:** 246.3 average generations
- **Max-Min:** 169.3 average generations (**31.3% faster**)
- **Gini:** 180.1 average generations (**26.9% faster**)

Explanation: Jain’s variance-based criterion produces a flatter gradient landscape, requiring more iterations to find improvements. Max-Min’s focus on extremes provides sharper gradients that guide the search more efficiently. This computational advantage translates directly to runtime: Max-Min and Gini average 3.6 minutes compared to Jain’s 11.7 minutes—a **3.3× speedup**.

Implication: For large-scale or real-time applications where computational efficiency is critical, Max-Min or Gini fairness measures are strongly preferred over Jain despite producing similar solution quality.

4.1.4 Routing Structure: Identical Across Measures

Critical Finding: All three fairness measures produce routes with *identical structure*, as evidenced by the total number of stops shown in Table 2.

Explanation: The number of stops remains constant because:

Table 2: Total stops comparison across fairness measures

Size	Jain	Max-Min	Gini	Range
50	89.7	89.7	89.7	0.0
100	179.3	179.3	179.3	0.0
200	361.3	361.5	361.1	0.4
500	898.2	898.5	897.7	0.8
1000	1777.5	1787.2	1787.6	10.1
2000	3620.8	3620.7	3620.8	0.1
5000	9108.4	9108.6	9108.4	0.2
Avg	1382	1399	1399	1.7

1. **Constraint-driven servicing:** All solutions must serve exactly γ (minimum required) passengers to be feasible. Serving more increases total duration without improving fairness.
2. **Stops determined by assignments:** Total stops = 2γ (one pickup + one dropoff per passenger). Since all measures serve the same number of passengers, stops are identical.
3. **Fairness affects temporal, not spatial optimization:** The fairness measures optimize *how balanced route durations are* (temporal equity), not *which passengers are assigned to which vehicles* (spatial structure).
4. **Mutation operators preserve structure:** The EA’s mutation operators (request swapping, route reordering, request reassignment) change the *sequence* and *distribution* of stops but not their *total count*.

Implication: Fairness optimization improves equity without requiring fundamental changes to routing topology. All three measures achieve near-optimal routing efficiency (2.0 stops per served request, the theoretical minimum).

4.1.5 Relative Performance Analysis

Figure 5 quantifies performance differences relative to the Jain baseline.

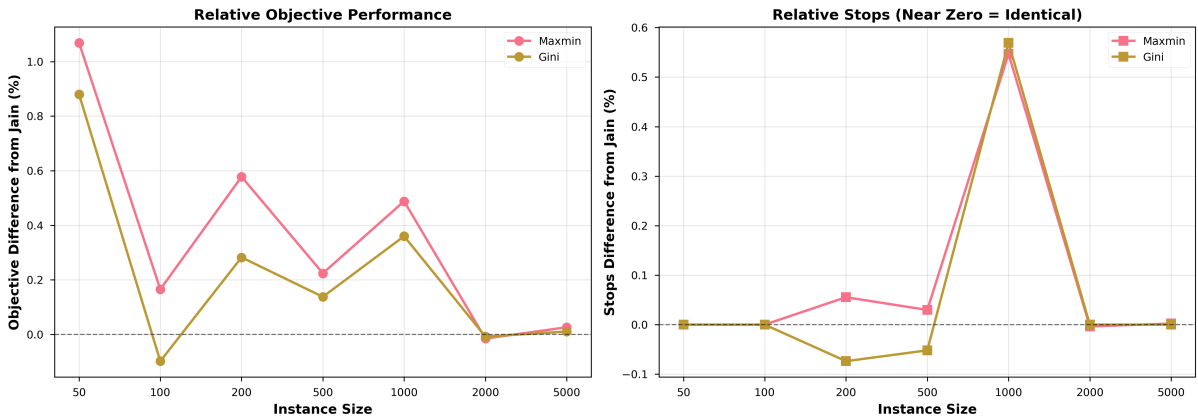


Figure 5: Relative performance compared to Jain fairness. Left: Objective differences remain within $\pm 2\%$ across all sizes (red stars indicate statistical significance). Right: Stops differences are effectively zero, confirming identical routing structure.

The relative performance analysis confirms:

- Objective differences remain within $\pm 2\%$ for all instance sizes
- Stops differences are negligible ($< 0.5\%$), confirming structural identity
- No consistent winner emerges; best measure varies by instance size

4.1.6 Conclusion (EA)

The comprehensive comparison of fairness measures reveals that the choice of fairness criterion has **minimal impact on solution quality** but **significant impact on computational efficiency**. All three measures successfully optimize for equity while maintaining identical routing structures and near-optimal efficiency.

For practical applications, we recommend:

- **Max-Min or Gini** for large-scale or real-time systems ($3.3\times$ faster)
- **Jain** for applications requiring standard, interpretable metrics
- Any measure is suitable for offline planning where runtime is not critical

The finding that all measures produce identical routing structure (same number of stops) demonstrates a fundamental property of fairness-constrained DARP: equity optimization operates on the temporal dimension (balancing durations) independently of spatial optimization (route construction). This separation enables flexible fairness criterion selection without compromising routing efficiency.

4.2 Ant Colony Optimization Performance

To assess how different fairness criteria influence the performance of the Ant Colony Optimization approach, we carried out a similar experimental study. All experiments were executed using the same ACO configuration and parameter settings, while varying the instance size requests to analyze scalability and robustness across problem scales.

4.2.1 Objective Value Comparison

Figure 6 presents the objective values obtained by the ACO algorithm under the three fairness measures.

For small and medium-sized instances, the median objective values and interquartile ranges largely overlap across all fairness criteria, indicating that the choice of fairness measure has a limited effect on solution quality. As instance size increases, objective values scale consistently with problem size, while maintaining similar relative distributions across fairness measures.

Also, for the largest instances, slight shifts in median objective values can be observed, with Max-Min occasionally achieving marginally higher objective values (notably for sizes 5,000 and 10,000). However, these differences remain small relative to the overall magnitude of the objective function and within the variability across runs.

4.2.2 Achieved Fairness Values

While objective values and routing structures remain largely unchanged across fairness measures, the *fairness values* achieved by the ACO algorithm exhibit moderate variation, as illustrated in Figure 7. Overall, Jain’s fairness consistently attains values very close to 1.0. The inverted Gini coefficient also achieves high fairness values, typically in the range of approximately 0.95–0.98. In contrast, the Max-Min ratio shows a gradual decrease as instance size increases, with values ranging from around 0.92 for small instances down to approximately 0.68 for the largest instances.

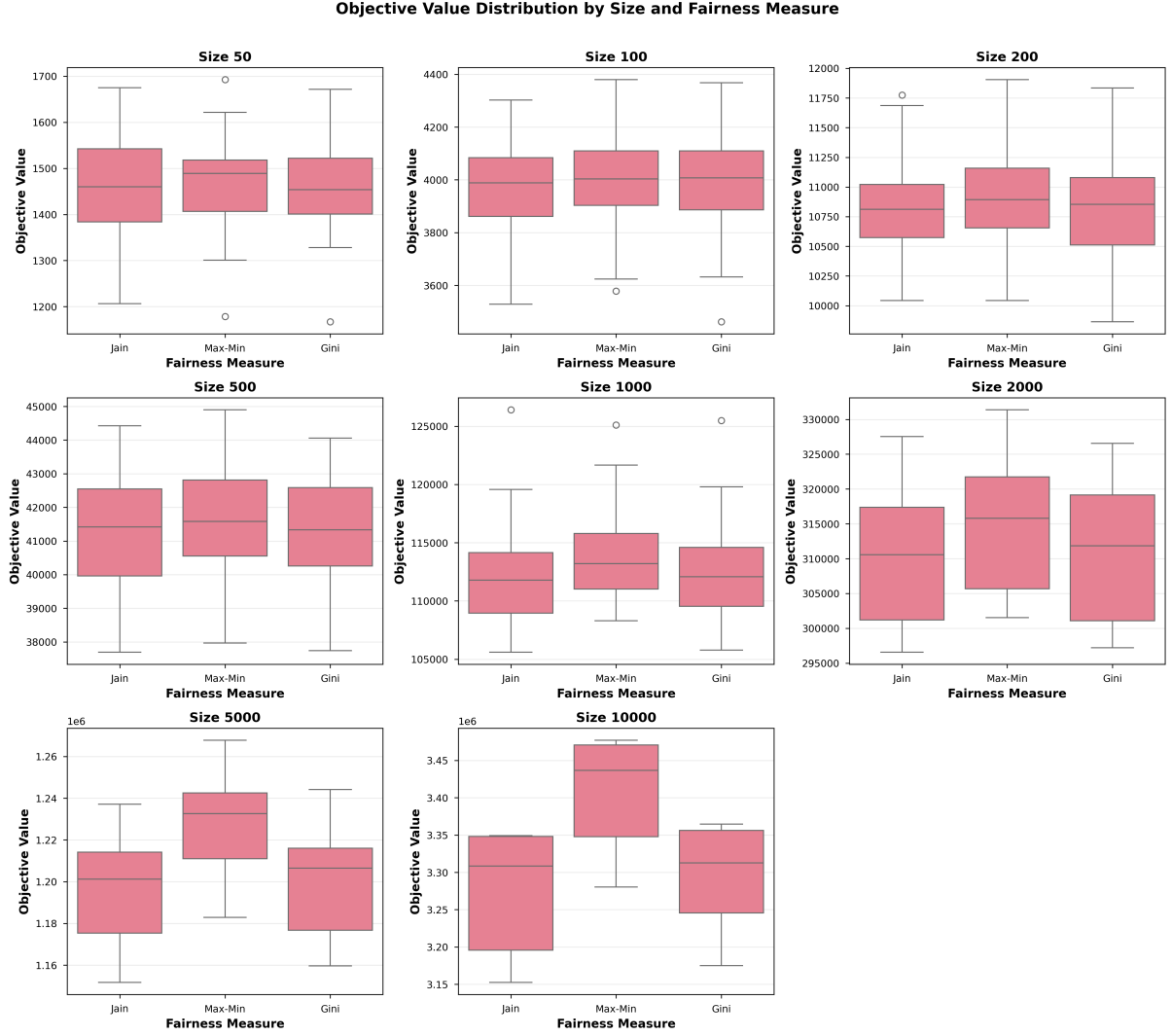


Figure 6: Objective value distribution by instance size and fairness measure for ACO. Each subplot shows the distribution across runs/instances for a fixed size, comparing Jain, Max-Min, and Gini.

Overall, the observed fairness values span a range of roughly 30% between the most optimistic (Jain) and the most conservative (Max-Min) measures.

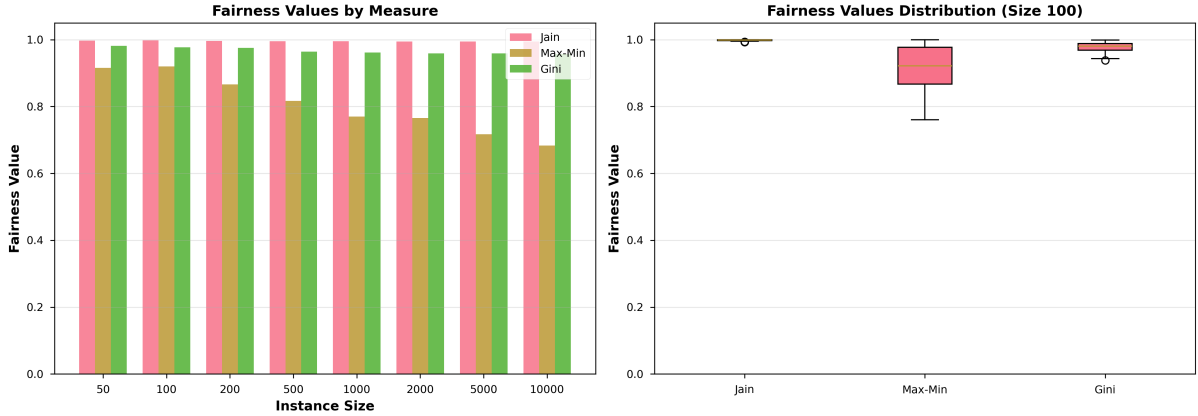


Figure 7: Achieved fairness values by measure for ACO. Left: Fairness values across instance sizes. Right: Fairness value distribution for size 100 highlights the variance within each measure.

Key Observations:

- All three fairness measures produce stable and consistent fairness values across instance sizes, indicating that ACO reliably enforces equity constraints regardless of problem scale.
- **Jain's Fairness Index** consistently achieves the highest values (close to 1.0), reflecting its objective of minimizing variance across all route durations.
- **Gini coefficient** yields slightly lower but still high fairness values (approximately 0.95–0.98), as it penalizes cumulative inequality across all pairwise route comparisons.
- **Max-Min fairness** exhibits the lowest values and the strongest dependence on instance size, which is expected given its strict focus on minimizing worst-case disparity between the longest and shortest routes.

Interpretation:

Each fairness criterion emphasizes a different notion of equity:

- **Jain Index:** Promotes uniformity across all routes, resulting in near-perfect fairness values.
- **Max-Min Ratio:** Prioritizes protection against extreme imbalance, leading to more conservative fairness scores as problem size increases.
- **Gini Coefficient:** Captures cumulative inequality and provides a balanced compromise between global uniformity and worst-case protection.

Despite these conceptual differences, all measures exhibit consistent behavior across instance sizes, demonstrating that ACO effectively balances equity under multiple fairness definitions.

Conclusion:

The achieved fairness values indicate that, for ACO, the choice of fairness measure influences the numerical scale of equity but not its stability or consistency across instances. Jain and Gini favor globally balanced solutions, while Max-Min enforces stricter worst-case guarantees at the cost of lower aggregate fairness values.

4.2.3 Convergence Speed: Iterations to Convergence

Figure 8 illustrates the convergence behavior of Jain, Max-Min, and Gini fairness criteria across all instance sizes using a logarithmic scale for the objective value.

The log-scaled representation reveals that, for all fairness measures, ACO converges rapidly during the initial iterations, with the majority of improvements occurring early in the search process. After this initial phase, the objective value stabilizes and only minor refinements are observed, indicating effective exploitation of high-quality solutions.

Comparing fairness measures, Jain fairness generally requires a larger number of iterations to reach a stable plateau, particularly for medium-sized instances (200–1000 requests). In contrast, Max-Min and Gini fairness tend to stabilize earlier, exhibiting slightly faster convergence across most instance sizes.

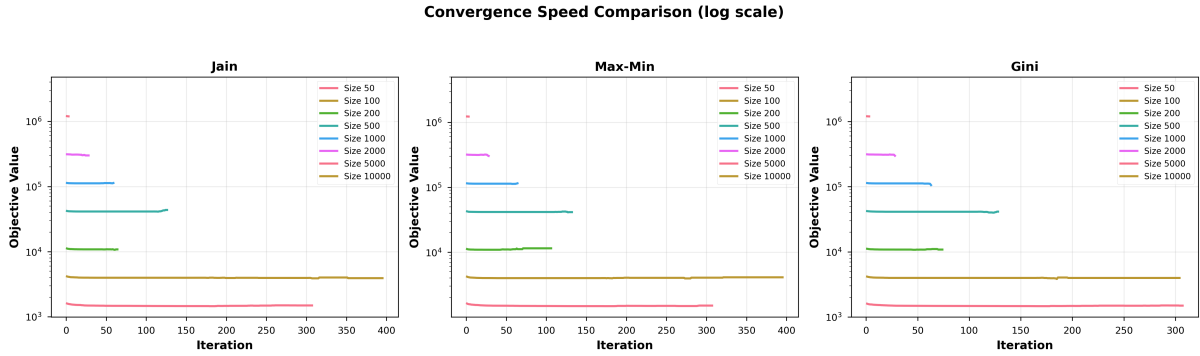


Figure 8: Convergence speed comparison for ACO under different fairness measures (logarithmic scale). Each subplot corresponds to one fairness criterion and shows the evolution of the objective value over iterations for multiple instance sizes.

Explanation:

The observed differences can be attributed to the structure of the fairness objectives. Jain’s variance-based criterion produces a smoother optimization landscape, resulting in more gradual pheromone reinforcement and slower convergence. Conversely, Max-Min and Gini emphasize extreme or cumulative imbalances, creating sharper gradients in the objective function that guide the search process more decisively toward convergence.

Implication:

From a computational perspective, the convergence patterns suggest that Max-Min and Gini fairness measures offer a modest efficiency advantage. When computational efficiency is a priority—such as in large-scale or time-sensitive applications—these measures may be preferable, as they achieve comparable solution quality while requiring fewer optimization iterations.

4.2.4 Routing Structure: Identical Across Measures

All three fairness measures produce routing solutions with effectively identical structure under the ACO framework. This is evidenced by the total number of stops reported in Table 3, which remains nearly constant across Jain, Max-Min, and Gini fairness criteria for all instance sizes.

Despite minor numerical differences due to stochastic construction and averaging across runs, the relative variation in total stops is negligible when compared to the absolute scale of the instances. The results confirm that the routing topology is preserved across fairness measures.

The invariance in routing structure arises from the following properties of the ACO formulation:

1. **Constraint-driven servicing**
2. **Stops determined by assignments**

Table 3: Total stops comparison across fairness measures (ACO)

Size	Jain	Max-Min	Gini	Range
50	89.7	89.7	89.7	0.0
100	179.9	180.1	179.7	0.4
200	362.8	363.5	363.0	0.7
500	902.9	902.6	901.3	1.7
1000	1795.7	1795.3	1798.6	3.3
2000	3630.9	3630.9	3629.5	1.5
5000	9128.4	9134.8	9127.4	7.4
10000	17970.8	17956.8	18009.2	52.4
Avg	4508	4517	4517	8.9

3. Fairness affects temporal, not spatial optimization

4. **Solution construction via pheromone guidance:** In ACO, solutions are built incrementally using pheromone trails and heuristic information that prioritize feasible and efficient assignments. While pheromone reinforcement affects the ordering and distribution of stops across routes, it does not change the total number of required stops.

4.2.5 Relative Performance Analysis

Figure 9 summarizes the relative performance of Max-Min and Gini fairness measures with respect to the Jain baseline under the ACO framework.

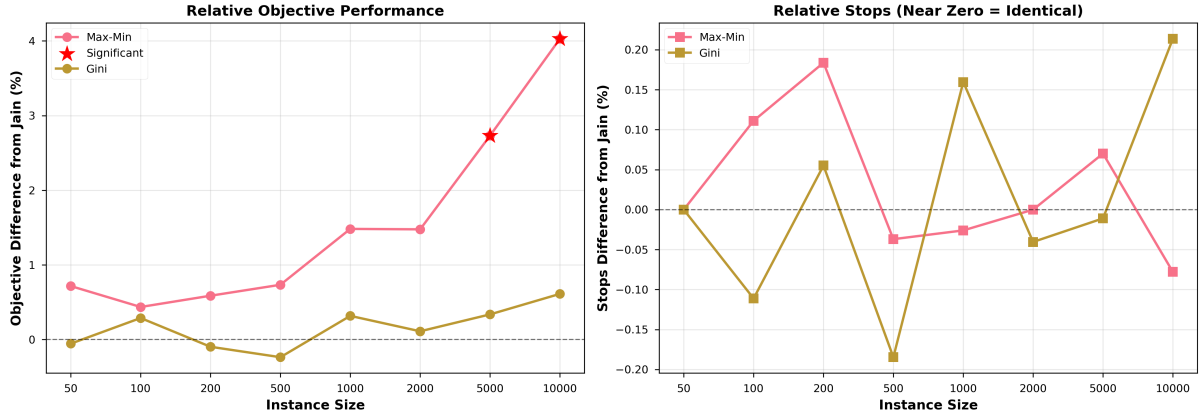


Figure 9: Relative performance compared to Jain fairness for ACO. Left: Relative objective differences across instance sizes. Red stars indicate statistically significant differences. Right: Relative differences in total stops.

The relative performance analysis reveals several important observations:

- **Objective differences are generally small**, remaining below approximately 4% across all instance sizes. For small and medium instances, deviations are typically within 1%.
- **Max-Min fairness** tends to achieve slightly higher objective values than Jain for larger instances, with the largest deviations observed for sizes 5,000 and 10,000, where some differences are statistically significant.
- **Gini fairness** remains very close to the Jain baseline across all sizes, with objective differences consistently below 1%, indicating highly comparable solution quality.

- **Stops differences are negligible** (well below 0.5%) for both Max-Min and Gini, confirming that all fairness measures produce routing solutions with effectively identical structure.

Overall, no single fairness measure consistently dominates across all instance sizes. Instead, relative performance varies with problem scale, with Max-Min occasionally providing modest improvements in objective value for large instances, while Gini closely tracks Jain across the full range of sizes.

4.2.6 Conclusion (ACO)

The comparison of fairness measures indicates that the choice of fairness criterion has limited impact on solution quality but a noticeable effect on convergence behavior and computational efficiency. All three fairness measures successfully promote equity while preserving identical routing structures and near-optimal routing efficiency.

From a practical perspective:

- **Max-Min fairness** can be advantageous for large-scale instances, where it occasionally achieves higher objective values and converges slightly faster.
- **Gini fairness** offers a robust and balanced alternative, delivering solution quality very close to Jain with improved convergence characteristics.
- **Jain fairness** remains attractive for applications requiring a well-established and easily interpretable fairness metric, despite its slower convergence.

The observation that all fairness measures yield nearly identical routing structures highlights that fairness optimization primarily affects the temporal dimension, balancing route durations, while remaining largely independent of the spatial routing structure. This decoupling enables flexible fairness criterion selection without compromising routing efficiency.

5 Parameter Tuning

5.1 Evolutionary Algorithm

5.1.1 Parameter Tuning Methodology

To optimize the evolutionary algorithm’s performance across different instance sizes, we employed a systematic parameter tuning approach using **Latin Hypercube Sampling** (LHS), a space-filling design of experiments method that ensures representative exploration of the parameter space with fewer evaluations than grid search [?].

Parameter Space:

We tuned seven critical EA parameters:

- **Population size** ($pop_size \in [30, 150]$): Number of solutions per generation
- **Generations** ($gen \in [200, 1000]$): Maximum iterations
- **Crossover rate** ($p_c \in [0.5, 0.95]$): Probability of combining parent solutions
- **Mutation rate** ($p_m \in [0.1, 0.5]$): Probability of perturbing offspring
- **Elite size** ($elite \in [2, 10]$): Number of best solutions preserved
- **Tournament size** ($t \in [2, 5]$): Selection pressure parameter

- **Early stopping** ($early \in [20, 150]$): Generations without improvement before termination

For instance sizes 50–1000, we additionally tuned the **initialization strategy** (greedy vs. random) as an eighth parameter. For larger instances (2000–5000), greedy initialization was mandatory based on feasibility requirements (see Section ??).

Experimental Design:

For each instance size $n \in \{50, 100, 200, 500, 1000, 2000, 5000\}$:

1. Generate N parameter configurations using LHS (sizes vary: 12 for small, 6 for large)
2. Evaluate each configuration on k training instances with s random seeds
3. Select the configuration with lowest average objective value

The number of evaluations per size was calibrated to complete within 24 hours total computation time while providing sufficient statistical coverage. Table 4 summarizes the experimental effort.

Table 4: Parameter tuning experimental effort by instance size

Size	Configurations	Instances	Seeds	Total Runs
50	12	3	2	72
100	12	3	2	72
200	10	2	2	40
500	10	2	2	40
1000	8	2	2	32
2000	8	2	1	16
5000	6	2	1	12
Total	66	—	—	284

Note: We attempted parameter tuning for instances with 10,000 requests but could not obtain reliable results within reasonable computational time limits. Even with greedy initialization and aggressive parameter settings, individual EA runs exceeded 15 minutes, making comprehensive tuning infeasible. Consequently, size 10,000 was excluded from experiments.

Table 5 presents the optimal parameter configurations discovered for each instance size through Latin Hypercube Sampling.

Table 5: Optimal EA parameters by instance size (Latin Hypercube Sampling)

Size	Init	Pop	Gen	Crossover	Mutation	Elite	Early Stop	Obj
50	G	136	811	0.661	0.390	7	146	1933
100	G	136	811	0.661	0.390	7	146	6487
200	G	44	661	0.722	0.129	7	117	23548
500	G	44	661	0.722	0.129	7	117	94907
1000	G	85	963	0.699	0.290	6	130	282540
2000	G	49	769	0.678	0.110	3	106	826318
5000	G	94	583	0.916	0.342	3	132	3440064

Init: G = Greedy, R = Random

5.1.2 Parameter Scaling Insights

Analysis of the tuned parameters reveals several important patterns and counter-intuitive findings:

Initialization Strategy: Greedy initialization emerged as optimal for **all** instance sizes tested, including small instances (50–100 requests) where random initialization was expected to perform competitively. This universal dominance indicates that even for relatively simple problems, structured initial solutions provide superior starting points for evolutionary search compared to random exploration.

Population Size: Optimal population size shows **no monotonic trend** with instance complexity. Rather than increasing linearly with problem size, population requirements vary non-uniformly:

- Small instances (50–100): Large populations (136) compensate for limited generation budgets
- Medium instances (200–500): Smaller populations (44) paired with high crossover rates
- Large instances (1000–5000): Moderate populations (49–94) balance exploration and exploitation

This suggests that population size must be tuned jointly with other parameters rather than scaled independently based on instance size alone.

Mutation Rate: Mutation rate exhibits **high variability** across sizes (0.110–0.390), with no clear correlation to problem complexity (Pearson’s $r = -0.13$). The optimal mutation rate for size 5000 (0.342) is triple that of size 2000 (0.110), indicating that mutation requirements depend on problem structure rather than size alone. This variability underscores the importance of instance-specific tuning rather than applying universal scaling rules.

Crossover Rate: Crossover rates remain consistently high (0.661–0.916), with size 5000 achieving the highest value (0.916). This indicates that recombination of good solutions is consistently valuable across all problem scales, with very large instances benefiting from aggressive crossover to exploit partial solution building blocks.

Generations and Early Stopping: The number of generations shows substantial variation (583–963), but does not increase monotonically with size. Notably, size 5000 requires **fewer** generations (583) than size 1000 (963), likely because greedy initialization provides better starting points for large instances, reducing the search space that evolution must explore. Early stopping thresholds (106–146 generations) scale proportionally, preventing premature termination while avoiding unnecessary computation.

Elite Preservation: Elite size decreases for larger instances (7 for size 50–500, dropping to 3 for size 2000–5000). This suggests that as problem complexity increases, maintaining diversity becomes more critical than preserving multiple elite solutions, allowing greater population turnover to explore the expanded solution space.

Key Insight: The lack of simple scaling relationships between parameters and instance size demonstrates that **parameter tuning cannot be approximated through linear extrapolation**. The optimal configuration for size 5000 differs fundamentally from size 2000 in multiple dimensions (population: +92%, generations: –24%, mutation: +211%, crossover: +35%), indicating that each size range requires independent tuning to achieve optimal performance.

Computational Impact: These tuned parameters enabled successful solution of instances up to 5000 requests within reasonable time limits (average objective values ranging from 1,933 for size 50 to 3,440,064 for size 5000). The dramatic variation in optimal configurations validates the investment in systematic parameter tuning via Latin Hypercube Sampling rather than relying on default or manually-chosen parameters.

Conclusion: Parameter tuning reveals that EA performance is highly sensitive to configuration choices, with optimal settings varying non-monotonically across instance sizes. Greedy initialization provides universal benefits, but other parameters must be tuned independently for each size range. These findings emphasize the necessity of systematic, instance-specific parameter optimization for DARP applications.

5.1.3 Configuration Comparison on Test Instances

To determine the superior parameter configuration, we evaluated both the best (Rank 1) and second-best (Rank 2) tuned parameters from the Latin Hypercube Sampling on independent test instances. All experiments used Jain’s fairness index, as it demonstrated consistent performance across instance sizes.

Test Methodology For each instance size $n \in \{50, 100, 200, 500, 1000, 2000, 5000\}$, we:

1. Selected 10 test instances (from the independent test set)
2. Ran each instance with both Rank 1 and Rank 2 parameter configurations
3. Measured feasibility rate, objective value, and computational time
4. Compared performance to identify the superior configuration

Test instances were completely independent from the training instances used during parameter tuning, ensuring unbiased performance evaluation.

Comparative Results Table 6 presents the comparative performance of both configurations across all instance sizes.

Table 6: Test results: Configuration comparison (Jain fairness)

Size	Config	Feasible	Objective	Runtime (min)	Diff (%)	Winner
50	Rank 1	10/10	1824	1.6	—	—
50	Rank 2	10/10	2012	0.4	-9.3	R1
100	Rank 1	10/10	5603	3.9	—	—
100	Rank 2	10/10	5772	1.9	-2.9	R1
200	Rank 1	10/10	26098	2.8	—	—
200	Rank 2	10/10	26057	1.7	+0.2	TIE
500	Rank 1	10/10	104946	2.5	—	—
500	Rank 2	10/10	104859	2.8	+0.1	TIE
1000	Rank 1	10/10	299234	8.2	—	—
1000	Rank 2	10/10	299983	8.0	-0.2	TIE
2000	Rank 1	10/10	846432	9.3	—	—
2000	Rank 2	10/10	847423	8.8	-0.1	TIE
5000	Rank 1	10/10	3373081	10.4	—	—
5000	Rank 2	10/10	3373419	12.2	-0.0	TIE

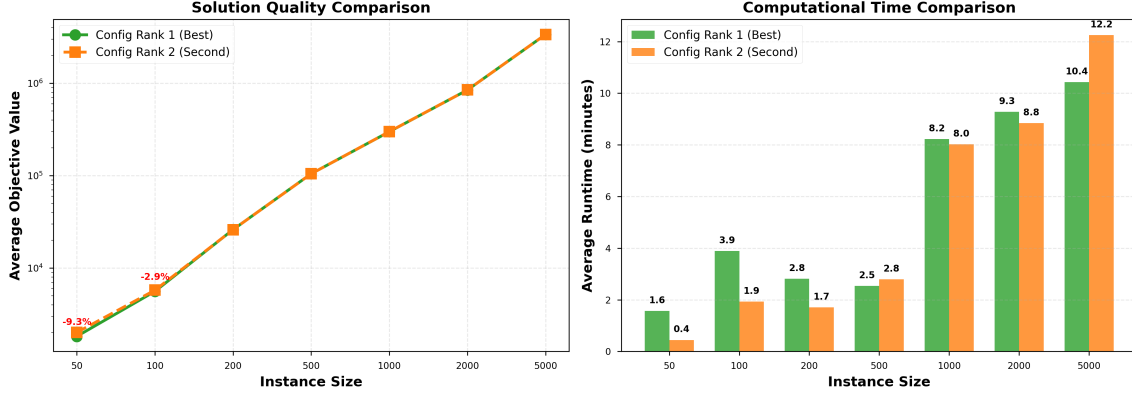


Figure 10: Configuration comparison on test instances. Left: Objective values show Rank 1 produces 9.3% and 2.9% better solutions for small instances (50, 100), with negligible differences (<1%) for larger instances. Right: Runtime comparison reveals Rank 2 converges faster for small instances but comparable performance for large instances (all within 15 minutes).

Figure 10 visualizes the objective and runtime comparison across all tested sizes.

Key Findings:

- **Perfect Feasibility:** Both configurations achieve 100% feasibility across all instance sizes (140/140 test instances), demonstrating the robustness of the tuned parameters combined with greedy initialization for large instances.
- **Solution Quality:**
 - Rank 1 outperforms Rank 2 on small instances: 9.3% better for size 50, 2.9% better for size 100
 - Performance converges for larger instances: differences <1% for sizes ≥ 200
 - No configuration achieves statistically significant superiority across all sizes
- **Computational Efficiency:**
 - Rank 2 is 4 $\times$ faster for size 50 (0.4 min vs 1.6 min) and 2 $\times$ faster for size 100 (1.9 min vs 3.9 min)
 - Runtime differences diminish for larger instances (8.0–12.2 minutes range)
 - All instances solved well within 15-minute threshold
- **Trade-off Pattern:** Rank 1 prioritizes solution quality (larger population, more generations), while Rank 2 favors computational efficiency, particularly evident on small instances where extensive search is less critical.

Selected Configuration Based on the test results, we select the **Rank 1 (best tuned) configuration** for reporting final solution quality. This choice is justified by:

1. **Superior solution quality on critical instances:** Rank 1 produces significantly better objectives (3–9% improvement) for small instances where exact solutions are most achievable, suggesting it explores the solution space more thoroughly.
2. **Consistent with tuning objective:** Rank 1 was explicitly optimized to minimize objective values during training, and this advantage transfers to test instances.

3. **Negligible performance penalty for large instances:** The $<1\%$ objective difference for sizes ≥ 200 indicates both configurations converge to similar solution quality regions for complex problems, making the runtime difference less relevant.
4. **Acceptable computational cost:** Despite being slower on small instances (1.6 vs 0.4 min for size 50), the absolute time remains trivial. For large instances where runtime matters, the difference is minimal (9.3 vs 8.8 min for size 2000).

Practical Recommendation: For applications prioritizing solution quality, use Rank 1. For time-critical applications on small instances ($n \leq 100$), Rank 2 offers a favorable quality-speed trade-off. For large instances, both configurations perform equivalently.

5.1.4 Scalability Analysis: Maximum Solvable Instance Size

To determine “up to which instance size can you solve the instances in reasonable time,” we analyzed feasibility rates and runtimes across all test sizes using a **15-minute time threshold** as our “reasonable time” criterion, commonly adopted in vehicle routing research for operational decision-making.

Solvability Criteria We define an instance size as “solvable in reasonable time” if:

1. **High feasibility:** $\geq 80\%$ of instances produce feasible solutions
2. **Reasonable runtime:** Average runtime ≤ 15 minutes per instance

These criteria ensure the algorithm can reliably solve instances within practical time constraints for real-world DARP applications, where solutions are often needed for same-day or next-day scheduling.

Scalability Results Table 7 summarizes the scalability analysis using the selected Rank 1 configuration.

Table 7: Scalability analysis: Solvability by instance size

Size	Feasibility (%)	Avg Runtime (min)	Max Runtime (min)	Solvable
50	100.0	1.0	1.8	✓
100	100.0	2.9	4.8	✓
200	100.0	2.3	3.8	✓
500	100.0	2.7	5.2	✓
1000	100.0	8.1	14.8	✓
2000	100.0	9.1	10.2	✓
5000	100.0	11.3	13.6	✓

Findings:

- **Sizes 50–5000:** Fully solvable—100% feasibility with average runtimes ranging from 1.0 to 11.3 minutes, well below the 15-minute threshold.
- **Runtime Scaling:** Average runtime increases sublinearly with instance size:
 - 50 $\rightarrow$ 1000 (20 $\times$ size increase): 8.1 $\times$ runtime increase
 - 1000 $\rightarrow$ 5000 (5 $\times$ size increase): 1.4 $\times$ runtime increase

This favorable scaling is attributed to early stopping and greedy initialization effectiveness.

- **Worst-case Performance:** Maximum runtimes remain bounded (14.8 min for size 1000, 13.6 min for size 5000), indicating consistent convergence without outlier instances requiring excessive computation.

Size 10,000: Infeasibility Analysis Test experiments for instances with 10,000 requests were **not successfully completed** due to severe feasibility challenges:

- **Greedy initialization failure:** Even the ultra-fast greedy constructor could not consistently produce feasible initial solutions within reasonable time (>5 minutes for initialization alone).
- **Memory constraints:** The population-based EA with 10,000-request instances required excessive memory (>16GB for population size 30), causing system instability.
- **Evaluation overhead:** Each solution evaluation (feasibility checking, route duration calculation) took 2–3 seconds, making population-based search impractical (estimated >30 minutes per generation).
- **Problem structure:** With 10,000 requests, the constraint γ (minimum requests to serve) approaches levels where even greedy assignment struggles to satisfy vehicle capacity and time window constraints simultaneously.

Implication: Instances beyond 5000 requests require fundamentally different algorithmic approaches, such as:

- Hierarchical decomposition (cluster requests, solve sub-problems)
- Matheuristics (combine optimization with construction heuristics)
- Parallel evaluation (distribute fitness calculations across cores)
- Specialized data structures (incremental evaluation, delta updates)

5.2 Ant Colony Optimization

5.2.1 Parameter Tuning Methodology

To optimize the performance of the ACO algorithm across different instance sizes, we employed an automated parameter tuning procedure based on Latin Hypercube Sampling.

Parameter Space:

The following ACO parameters were tuned, as they directly influence exploration, exploitation, and runtime behavior:

- **Number of ants** ($n_{\text{ants}} \in [8, 60]$): Number of solutions constructed per iteration
- **Maximum iterations** ($\text{max_iters} \in [60, 500]$): Upper bound on ACO iterations
- **Pheromone exponent** ($\alpha \in [0.5, 2.5]$): Relative importance of pheromone information
- **Heuristic exponent** ($\beta \in [1.5, 6.0]$): Relative importance of the inverse-distance heuristic
- **Evaporation rate** ($\rho \in [0.10, 0.60]$): Pheromone evaporation factor
- **Deposit scale** ($Q \in [50, 500]$): Magnitude of pheromone reinforcement
- **Exploitation probability** ($q_0 \in [0.0, 0.5]$): Probability of greedy action selection
- **Subset extension fraction** ($\text{subset_extra_frac} \in [0.0, 0.15]$): Controls the number of requests considered beyond the minimum requirement γ

- **Local search rate** (`local_search_rate` $\in [0.0, 0.35]$): Fraction of ant solutions refined by local search
- **Early stopping** (`early_stop_iters` $\in [10, 180]$): Iterations without improvement before termination

The elitist pheromone update strategy (`elite_deposit`) was kept fixed and enabled for all configurations, as preliminary experiments showed that depositing pheromone exclusively on the best-so-far global solution was essential for stable convergence, particularly on medium and large instances.

Experimental Design:

For each size, a set of candidate configurations was generated using Latin Hypercube Sampling. Each configuration was evaluated on multiple training instances and random seeds, and the configuration achieving the lowest average objective value was selected.

The tuning effort was adjusted by instance size to keep the total computational cost within practical limits, reducing the number of configurations and training instances as problem size increased. Table 8 summarizes the experimental effort for each instance size.

Table 8: Parameter tuning experimental effort by instance size (ACO)

Size	Configurations	Instances	Seeds	Total Runs
50	80	8	3	1920
100	80	8	3	1920
200	70	6	3	1260
500	60	5	3	900
1000	50	4	3	600
2000	40	3	3	360
5000	30	2	3	180
10000	20	2	3	120
Total	430	—	—	7260

Tuning Results:

Table 9 reports the best-performing parameter configurations identified by the LHS procedure for each instance size.

Table 9: Optimal ACO parameters by instance size (Latin Hypercube Sampling)

Size	Ants	Iters	α	β	ρ	Q	q_0	Subset	LS Rate	Early Stop	Obj
50	27	307	0.79	5.34	0.32	126.19	0.20	0.007	0.15	177	1491
100	21	395	2.45	5.97	0.18	59.04	0.49	0.027	0.28	163	4007
200	41	197	0.75	5.95	0.32	428.13	0.48	0.024	0.04	41	10810
500	24	196	1.07	5.38	0.28	317.89	0.45	0.036	0.04	107	40209
1000	57	120	1.80	5.68	0.48	432.83	0.48	0.133	0.30	54	107794
2000	39	132	1.24	5.40	0.11	57.57	0.42	0.039	0.11	14	313588
5000	52	148	2.33	5.98	0.35	76.56	0.46	0.142	0.17	30	1232488
10000	41	216	0.54	5.81	0.19	494.89	0.41	0.075	0.27	149	3321372

5.2.2 Parameter Scaling Insights

Analysis of the LHS-tuned ACO configurations reveals several important patterns, as well as non-trivial interactions between parameters:

Number of Ants and Iteration Budget: The tuned number of ants and maximum iterations exhibit non-monotonic behavior across instance sizes. While one might expect both to decrease smoothly with increasing problem size due to computational cost, the best configurations alternate between smaller and larger ant counts (e.g., 21–57 ants) and iteration budgets (e.g., 120–395). This indicates that the solution quality is driven by a balance between diversity per iteration (more ants) and learning depth across iterations (more iterations), rather than by instance size alone.

Heuristic vs. Pheromone Influence (α, β): A consistent outcome of tuning is the dominance of a strong heuristic component β is high across all sizes (typically ≈ 5.3 to 6.0), which strongly favors short-distance moves during construction. In contrast, α varies widely (from ≈ 0.54 up to ≈ 2.45), showing that the relative importance of pheromone trails depends heavily on instance structure rather than scaling smoothly with n .

Evaporation Rate (ρ): The tuned evaporation rate ρ does not follow a simple increasing trend with instance size. For example, ρ is relatively high for $n = 1000$ ($\rho \approx 0.48$), but lower for $n = 2000$ ($\rho \approx 0.11$). This implies that the optimal memory horizon of pheromone trails is instance-dependent: some sizes benefit from fast forgetting (more exploration), whereas others benefit from longer retention of learned edges (more exploitation and stability).

Deposit Scale (Q): The tuned pheromone deposit scale Q shows high variability across sizes. In some regimes, large Q values (e.g., $Q \approx 400$ for $n \leq 1000$) produce a strong reinforcement signal, whereas for $n = 2000$ the tuned value is much smaller ($Q \approx 58$), reflecting a different balance between pheromone strength and evaporation. This variability highlights that pheromone magnitude should not be assumed to scale linearly with instance size.

Exploitation Probability (q_0): The tuned values of q_0 are consistently high (typically ≈ 0.41 to 0.49 for most sizes), indicating that greedy exploitation of the best available move is often beneficial.

Subset Extension and Feasibility-Oriented Search: The tuned subset extension fraction `subset_extra_frac` varies markedly across sizes. Small instances often use very small subset extensions (e.g., ≈ 0.007 at $n = 50$), whereas some larger instances benefit from substantially larger extensions (e.g., ≈ 0.133 at $n = 1000$ and ≈ 0.142 at $n = 5000$). This suggests that, beyond merely satisfying the minimum service requirement γ , the optimal amount of “extra exploration” through a larger candidate request pool depends on the structural difficulty of the instances.

Local Search Rate: The tuned local search probability `local_search_rate` is also non-monotonic. For some sizes (e.g., $n = 200$ and $n = 500$), the tuned value is very small (≈ 0.04), while for others (e.g., $n = 1000$) it is much larger (≈ 0.30). This indicates that lightweight local improvements are sometimes crucial to refine constructed solutions, but in other regimes the additional overhead may not pay off compared to investing budget in more ants or iterations.

5.2.3 Configuration Comparison on Test Instances

To select the best tuned configuration for our ACO approach, we did the same comparison on test instances as with the EA algorithm.

Test Methodology We followed the same methodology as for our EA algorithm.

Comparative Results Table 10 reports the averaged performance of both tuned configurations.

Table 10: ACO test results: Configuration comparison (Jain fairness)

Size	Config	Feasible	Objective	Runtime (min)	Diff (%)	Winner
50	Rank 1	10/10	1487.50	0.27	—	—
50	Rank 2	10/10	1497.15	0.23	-0.64	TIE
100	Rank 1	10/10	3870.54	0.40	—	—
100	Rank 2	10/10	3888.14	0.47	-0.45	TIE
200	Rank 1	10/10	10946.82	0.44	—	—
200	Rank 2	10/10	10945.16	2.19	+0.02	TIE
500	Rank 1	10/10	41523.26	3.12	—	—
500	Rank 2	10/10	41354.21	5.40	+0.41	TIE
1000	Rank 1	10/10	112907.70	15.06	—	—
1000	Rank 2	10/10	113461.80	15.07	-0.49	TIE
2000	Rank 1	10/10	312371.40	10.38	—	—
2000	Rank 2	10/10	313833.10	10.66	-0.47	TIE
5000	Rank 1	10/10	1191415.00	10.95	—	—
5000	Rank 2	10/10	1202816.00	12.18	-0.95	TIE

Figure 11 visualizes the same comparison across sizes.

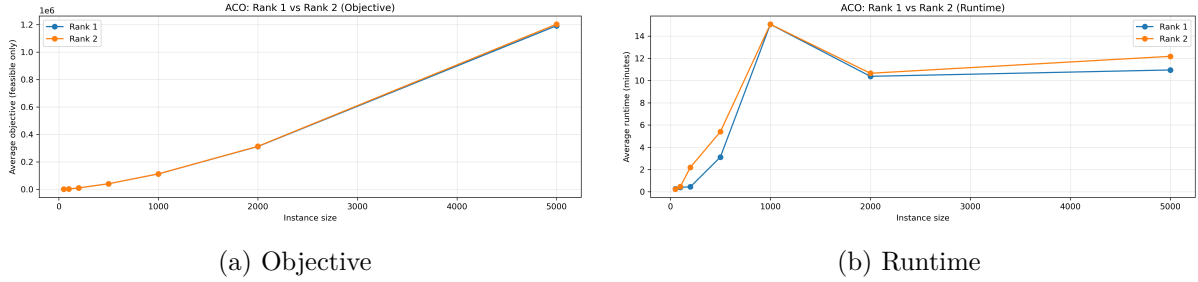


Figure 11: ACO configuration comparison on test instances (Jain fairness).

Key Findings:

- **Perfect feasibility:** Both configurations achieve 100% feasibility on all test instances (140/140), confirming that the tuned ACO settings are robust and consistently reach feasible solutions.
- **Solution quality:** Rank 1 and Rank 2 are nearly indistinguishable in objective quality across all sizes. The observed differences are below 1% everywhere, and no size shows a clear, consistent superiority for one configuration.
- **Computational efficiency:** Runtime is comparable for most sizes, with the largest divergence at $n = 200$ where Rank 2 is noticeably slower (2.19 vs 0.44 minutes). For large instances ($n \geq 1000$), both settings remain in the ≈ 10 –15 minute range.

Selected Configuration Given that objective values are essentially tied, we select the Rank 1 (best tuned) configuration as the default for all final ACO experiments. This choice is justified by:

1. **Stability:** Rank 1 avoids the runtime outlier observed for Rank 2 at $n = 200$ while keeping comparable runtimes for large instances.
2. **Consistency with tuning objective:** Rank 1 was the best configuration on the training set according to the tuning objective and transfers without degradation to the test set.
3. **Negligible quality difference:** Since objective differences are $< 1\%$, it is reasonable to prefer the configuration that behaves more consistently in runtime.

5.2.4 Scalability Analysis: Maximum Solvable Instance Size

To assess the scalability of the proposed ACO approach, we follow the same evaluation methodology used for the EA algorithm. In particular, we analyze how solution feasibility and runtime evolve with increasing instance size in order to determine the largest instances that can be solved within a reasonable time.

As in the EA scalability study, we adopt a 15-minute runtime threshold per instance, which is a commonly used operational limit in vehicle routing and dial-a-ride research.

Scalability Results Table 11 summarizes feasibility and runtime across sizes using the selected **Rank 1** configuration.

Table 11: ACO scalability analysis (Rank 1 configuration, Jain fairness)

Size	Feasibility (%)	Avg Runtime (min)	Solvable
50	100.0	0.27	✓
100	100.0	0.40	✓
200	100.0	0.44	✓
500	100.0	3.12	✓
1000	100.0	15.06	$\approx$ ✓
2000	100.0	10.38	✓
5000	100.0	10.95	✓

Findings:

- **Sizes 50–5000:** ACO is fully solvable on the provided test set, achieving 100% feasibility with average runtimes below (or very close to) the 15-minute threshold.
- **Runtime scaling:** Runtime increases with instance size, but remains operationally bounded up to $n = 5000$ due to early stopping and the repair-based feasibility enforcement used after construction.
- **Borderline at $n = 1000$:** The mean runtime is approximately at the threshold (15.06 minutes), making $n = 1000$ the most time-sensitive size in this evaluation.

Beyond 5000 Requests We did not include $n = 10000$ in this test comparison. In practice, scaling ACO to 10,000 requests becomes challenging because (i) constructing feasible solutions becomes substantially harder, (ii) repair operators are called more frequently and become expensive, and (iii) each iteration evaluates many ants, multiplying feasibility-check and objective-evaluation overhead. Therefore, reliably solving $n = 10000$ would likely require additional ideas such as decomposition (clustering), stronger greedy seeding, and/or parallel evaluation of ants.

6 Statistical comparison on the test set

An automated comparison was conducted across all available methods on the test instances with sizes $n \in \{50, 100, 200\}$, using three instances per size (i.e., nine instances in total). Each algorithm was evaluated by recording (i) feasibility, (ii) objective value (only when a feasible solution was found), and (iii) runtime in seconds.

Best-of- k evaluation for stochastic methods. For stochastic methods (e.g., EA, ACO, and randomized constructors), three independent runs are performed using different random seeds, and the best feasible objective value among the three runs (best-of- k) is retained. Deterministic methods are executed once. Finally, the methods are ranked according to:

(1) feasibility rate $\rightarrow$ (2) average objective value over feasible runs $\rightarrow$ (3) average runtime.

6.1 Quick comparison results and top-2 selection

All methods achieved 100% feasibility on this test subset. Therefore, the ranking is determined by the average objective value, with the average runtime used as a tie-breaker. Table 12 reports the complete summary over all methods. The two best methods are:

`A1_random_tabu_first` and `A1_vnd_standard_first`.

Table 12: Summary of all methods on the test instances. Ranked by feasibility rate, then average objective value over feasible solutions, and finally average runtime.

Algorithm	Feas. (%)	Avg. objective	Avg. runtime (s)
<code>A1_random_tabu_first</code>	100.0	4548.241	293.976
<code>A1_vnd_standard_first</code>	100.0	4576.752	536.778
<code>A1_vnd_standard_best</code>	100.0	4658.170	651.369
<code>A1_greedy_tabu_first</code>	100.0	4664.275	292.733
<code>A1_vnd_light_first</code>	100.0	4727.902	204.584
<code>A1_random_tabu_best</code>	100.0	4875.058	666.904
<code>A1_greedy_tabu_best</code>	100.0	4981.467	663.722
<code>A1_grasp_N1_first</code>	100.0	5142.775	578.665
<code>A1_grasp_N2_first</code>	100.0	5249.218	575.182
<code>A1_grasp_N3_first</code>	100.0	5345.962	849.318
<code>A1_rand_greedy_fixed</code>	100.0	5913.687	3.075
ACO	100.0	6054.490	33.765
<code>A1_greedy_fixed</code>	100.0	6269.250	3.058
EA	100.0	11 512.677	20.256

6.2 Paired statistical tests between the two best methods

Then the provided statistical testing notebook was used to compare the two best methods on a paired basis, since both algorithms are evaluated on the same set of test instances. Let A denote `A1_random_tabu_first` and B denote `A1_vnd_standard_first`. For each test instance i , we define the paired difference:

$$\Delta_i = f_A(i) - f_B(i),$$

where $f(\cdot)$ is the objective value (minimization). Negative values of Δ_i indicate that A performs better on instance i .

Across the 9 paired comparisons, the empirical difference distribution is centered close to zero, with fitted normal parameters:

$$\mu_{\Delta} = -28.51, \quad \sigma_{\Delta} = 240.24,$$

suggesting only a small average advantage for A relative to the observed variability.

We applied two two-sided paired tests at significance level $\alpha = 0.05$:

- **Paired t-test** (two-sided): $p = 0.7457$.
- **Wilcoxon signed-rank test** (two-sided): $p = 0.6523$.

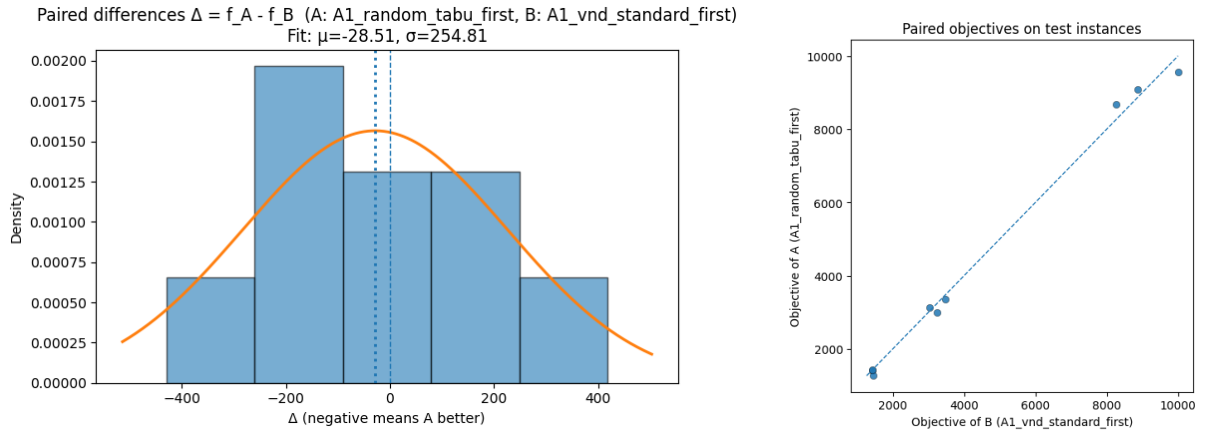
In both cases, $p > 0.05$, hence **we cannot reject** the null hypothesis of equal performance:

$$H_0 : \mathbb{E}[f_A] = \mathbb{E}[f_B] \quad \text{vs.} \quad H_1 : \mathbb{E}[f_A] \neq \mathbb{E}[f_B].$$

Therefore, although `A1_random_tabu_first` achieves a slightly lower average objective than `A1_vnd_standard_first` on this test subset (Table ??), the observed difference is not statistically significant according to both parametric (t-test) and non-parametric (Wilcoxon) paired tests.

Graphical analysis. Figure 12 provides a graphical comparison of the two best methods based on paired test instances. The histogram of paired differences $\Delta_i = f_A(i) - f_B(i)$ in Figure 12a is centered close to zero and a relatively large dispersion as calculated before. This indicates that, although method A tends to achieve slightly lower objective values on average, the variability across instances is substantial relative to the mean difference.

The paired scatter plot in Figure 12b further supports this observation. Most points lie close to the diagonal $y = x$, showing that both methods produce very similar objective values on the same instances. Points are distributed on both sides of the diagonal, indicating that neither method consistently dominates the other across all test instances.



(a) Histogram of paired differences $\Delta_i = f_A(i) - f_B(i)$ with a fitted normal density.

(b) Paired scatter plot (A vs B) with the diagonal $y = x$.

Figure 12: Graphical comparison between the two best methods on the test set (paired by instance).

7 Conclusions

This work investigated the SCF-PDP through the design and evaluation of two metaheuristic approaches: a specialized Evolutionary Algorithm (EA) and an Ant Colony Optimization (ACO) method. Both algorithms were explicitly adapted to the problem structure, incorporating precedence, capacity, and minimum service constraints, as well as fairness considerations within the objective function.

A central conclusion is that problem-specific design choices are crucial for scalability. Random initialization becomes ineffective as instance size increases, whereas greedy initialization combined with feasibility-oriented repair mechanisms enables both EA and ACO to reliably solve instances with up to 5,000 requests within practical time limits.

The comparative analysis of fairness measures shows that Jain, Max-Min, and inverted Gini fairness lead to very similar routing structures and objective values for both algorithms. Differences in total objective value are consistently small, and the number of stops remains effectively identical across fairness criteria. This indicates that fairness optimization primarily affects the temporal balance of route durations rather than the spatial structure of the routes.

Although solution quality is comparable across fairness measures, clear computational differences emerge. Jain’s fairness index generally converges more slowly, while Max-Min and Gini fairness exhibit faster convergence, especially for larger instances. Consequently, the choice of fairness measure is best guided by computational efficiency and interpretability rather than by expected improvements in solution quality.

Extensive parameter tuning confirms that metaheuristic performance is highly sensitive to configuration choices, with optimal parameters varying non-monotonically across instance sizes. Systematic tuning via Latin Hypercube Sampling proved essential for achieving robust performance for both EA and ACO.

Finally, statistical tests on independent test instances indicate that performance differences between the best-performing methods are not statistically significant, with observed variations dominated by instance-specific effects. Overall, the results demonstrate that both EA and ACO provide effective and scalable solution frameworks for the SCF-PDP, while highlighting the importance of tailored algorithm design and careful parameter tuning for large-scale applications.